

1. Architecture Globale du Système

Ce projet implémente un simulateur multi-agents de véhicules guidés automatisés (AGV) avec une architecture de **Jumeau Numérique** découplée. Le système est composé d'un moteur physique synchrone en Python (Backend) et d'un client de visualisation 3D réactif en React + Three.js (Frontend).

A. Diagramme d'Architecture Système

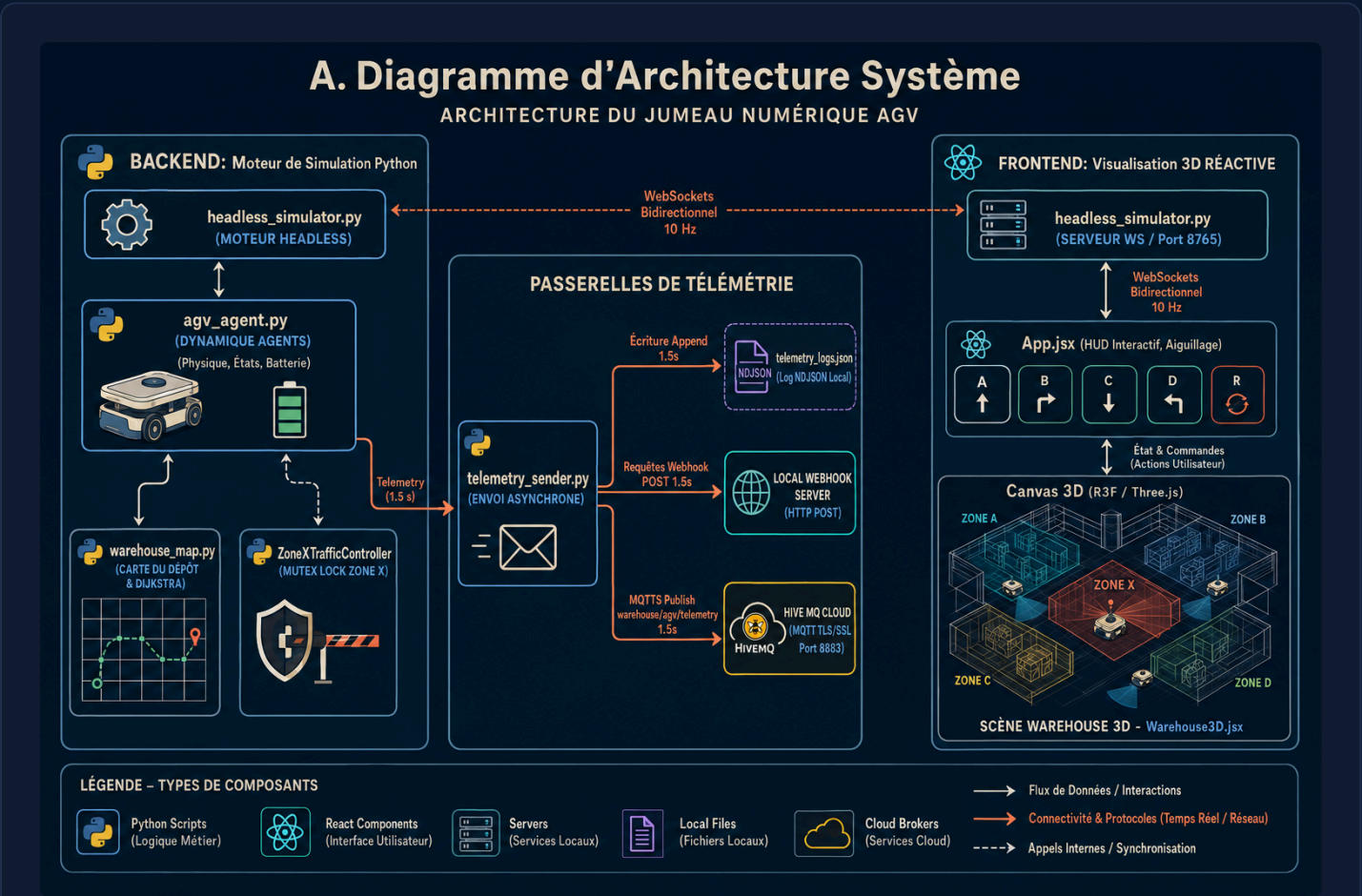


Figure 1.1 : Structure décentralisée de la simulation physique (Python) et du client 3D (React).

B. Diagramme de Flux de Données (Séquence)

B. Diagramme de Flux de Données (Séquence)

DIAGRAMME DE SÉQUENCE DES MESSAGES 10 HZ ET 1.5 S

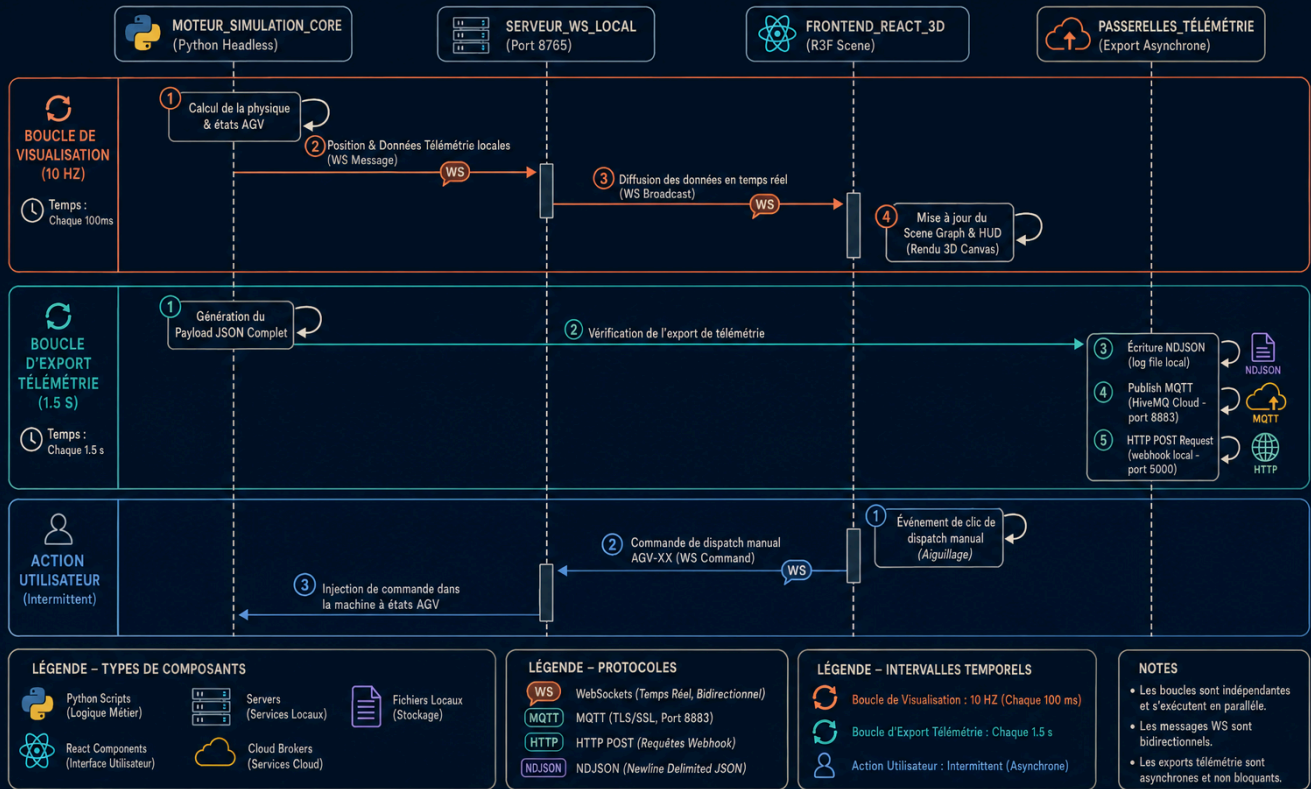


Figure 1.2 : Séquence d'échange des messages de télémétrie et des commandes manuelles d'aiguillage.

2. Composants du Backend (Python)

Le dossier comprend plusieurs modules Python assurant la simulation physique, le routage spatial, la prévention des collisions et l'envoi de la télémétrie :

A. Gestion des Agents AGV (agv_agent.py)

Ce fichier définit la classe `AGV` qui gère de manière autonome chaque robot :

- **Machine à États** : Les agents changent d'état dynamiquement selon un automate fini (`IDLE`, `EN_ROUTE`, `ARRIVED`, `WAIT`, `STOP`).
- **Physique de la Batterie** : Consommation d'énergie continue lors des déplacements (décharge complète en ~3 minutes d'activité) et recharge rapide (augmentation de 8% par seconde) une fois arrivé à la station de charge **ZONE R**.
- **Physique Thermique** : Échauffement du moteur sous charge (température max limitée à 48.5°C) et refroidissement progressif lors des arrêts ou de la charge.
- **Perte de Connexion Réaliste** : Simule les micro-coupures Wi-Fi des environnements industriels de manière asymétrique (taux d'indisponibilité global de 0.3%, avec reconnexion automatique en moins de 0.5s), permettant à l'AGV de continuer sa navigation physique locale en autonomie (Edge Computing).

B. Graphe & Routage Dijkstra (warehouse_map.py)

- **Réseau de Voies** : Représentation du dépôt sous forme de graphe (nœuds et arêtes interconnectant les zones de chargement A, de déchargement B, de tri C, d'emballage D, de recharge R, et le carrefour central X).
- **Pénalité Dijkstra** : Pour inciter les AGVs à privilégier l'autoroute centrale (nœud X) tout en laissant la possibilité d'utiliser les voies externes en cas de blocage, un coefficient de pénalité de **1.6** est appliqué aux arêtes périphériques.

C. Contrôleur de Trafic Exclusion Mutuelle (Mutex)

La **ZONE X** centrale est une intersection critique qui ne peut accueillir qu'un seul AGV à la fois.

- Le système implémente un verrou de synchronisation thread-safe (`ZoneXTrafficController`) faisant office de **Mutex**.
- L'AGV arrivant aux portes d'accès (nœuds d'entrée `X_N`, `X_S`, `X_W`, `X_E`) demande l'autorisation d'entrer. S'il l'obtient, il traverse. Sinon, il s'arrête instantanément en état `WAIT` jusqu'à libération de la zone.

D. Capteurs & Évitement de Collisions

- Chaque AGV possède un cône virtuel de détection de distance frontale de 45° (`distance_front_cm`).
- Si un AGV suiveur détecte un obstacle ou un autre AGV devant lui à moins de **80 cm**, il effectue un arrêt de sécurité (`STOP`).
- Le système filtre et ignore les AGVs circulant en sens inverse (grâce à un décalage géométrique automatique vers la droite de chaque sens de circulation), évitant ainsi les situations de blocage mutuel (deadlock).

E. Multi-Passerelles de Télémétrie (telemetry_sender.py)

Toutes les 1.5 secondes, les données physiques de chaque robot sont envoyées de manière asynchrone via trois canaux parallèles :

- **Fichier JSON local** : Écrit les entrées au format NDJSON dans `telemetry_logs.json` .
- **Webhook HTTP** : Envoie une requête POST contenant le payload JSON vers le serveur récepteur local `webhook_server.py` .
- **Broker MQTT Cloud** : Se connecte de manière sécurisée en TLS (port 8883) à un broker **HiveMQ Cloud**, et publie les données sur le canal `warehouse/agv/telemetry` .

3. Interface du Jumeau Numérique 3D (React + R3F)

Situé dans le dossier `/frontend`, l'application Web communique avec le simulateur Python via un canal WebSocket à **10 Hz** (10 trames/seconde) pour un rendu 3D fluide et temps réel.

A. Scene Graph 3D (Warehouse3D.jsx)

Rendu matériel sous React Three Fiber (Three.js) :

- Le dépôt est représenté sur un plan 3D avec grillage technique.
- Les zones physiques (A, B, C, D, R) sont matérialisées par des plaques colorées translucides rétroéclairées.
- La **ZONE X** centrale est mise en valeur par un contour épais Terracotta et des bandes hachurées d'avertissement.
- Les AGVs sont modélisés par des blocs robotiques 3D texturés avec des roues pivotantes et un cône lumineux de capteur qui vire au Terracotta clignotant lors du transit dans la ZONE X.
- Les caméras interactives (`OrbitControls`) permettent de pivoter (clic gauche glissé), déplacer le plan (clic droit glissé) et zoomer (molette).

B. Tableau de bord HUD interactif (App.jsx)

- **Indicateurs de Performance** : Cartes de télémétrie dotées de barres de progression plates pour la batterie et la vitesse, affichage de la température moteur, de l'odomètre accumulé et de l'état de connexion.
- **Alerte Flash Critique** : Un bandeau d'avertissement rouge clignotant apparaît immédiatement sur la carte de l'AGV dès qu'il pénètre physiquement dans la zone d'intersection centrale X.
- **Commandes Système** : Bouton **PAUSE/RESUME** gèle la boucle physique côté serveur Python tout en maintenant la connexion réseau active. Le bouton **STOP FLEET / RESUME FLEET** déclenche l'arrêt d'urgence général (E-Stop).
- **Aiguillage Manuel (Manual Dispatch)** : Chaque carte AGV dispose d'une série de boutons `[A]`, `[B]`, `[C]`, `[D]`, `[R]`. Cliquer sur un bouton envoie une commande WebSockets dédiée, forçant instantanément l'AGV sélectionné à planifier une nouvelle trajectoire et à se rendre vers la destination choisie par l'opérateur.

C. Capture d'Écran de l'Interface Utilisateur 3D

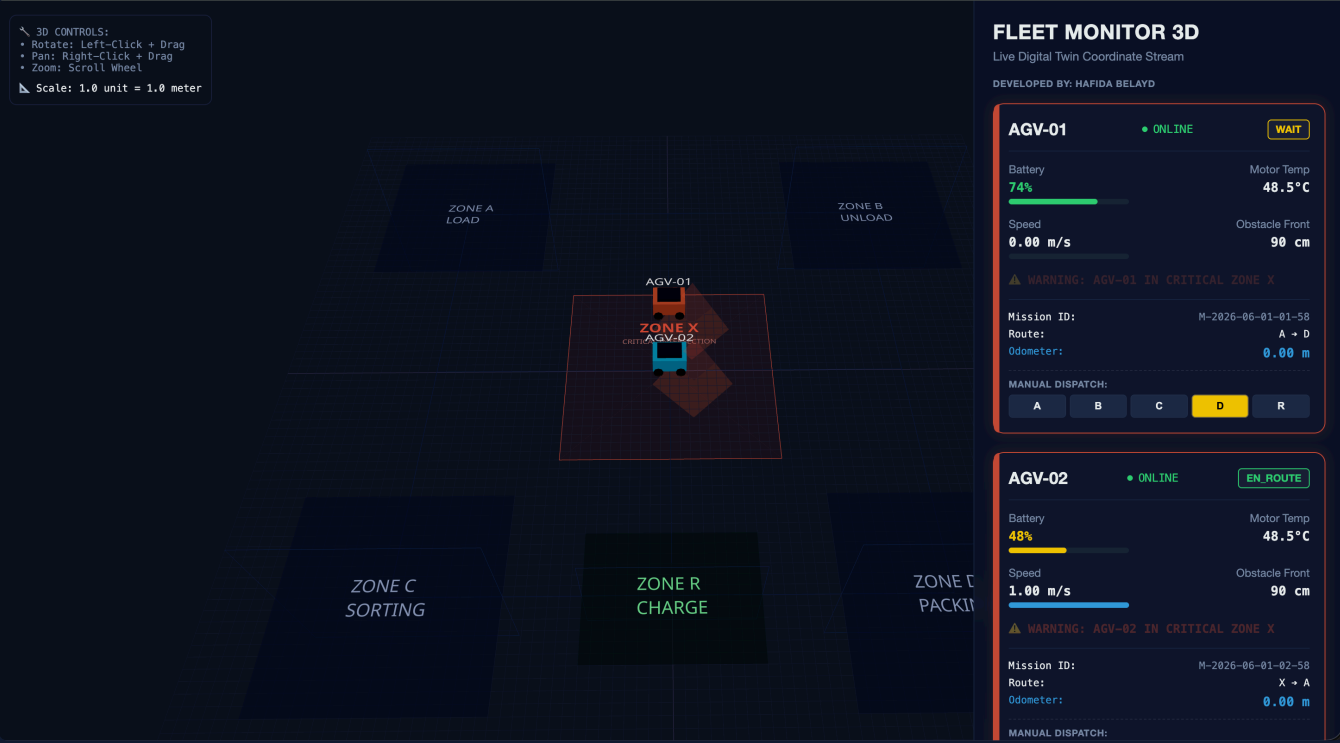


Figure 3.1 : Rendu en temps réel de l'IHM du jumeau numérique 3D avec les cartes HUD et les boutons d'aiguillage.

4. Tableau Récapitulatif des Protocoles de Communication

Protocole	Couche Transport	Fréquence / Débit	Format	Rôle dans le Système	Sécurité & Fiabilité
WebSockets	TCP (Port 8765)	10 Hz (100ms)	JSON	Synchronisation bidirectionnelle haute fréquence entre le backend et le jumeau numérique 3D.	Connexion locale persistante. Reconnexion automatique côté client.
MQTT (MQTTS)	TCP (Port 8883)	Intermittente (1.5s)	JSON Payload	Exportation de la télémétrie vers le cloud (HiveMQ) pour la supervision à distance.	Sécurisé via TLS/SSL. Gestion de la QoS (Quality of Service).
HTTP POST	TCP (Port 5000)	Intermittente (1.5s)	JSON Payload	Ingestion des données de routage vers des serveurs d'automatisation externes (n8n, Make).	Mode sans état (Stateless). Retries automatiques programmés.
NDJSON	Fichier Local	Intermittente (1.5s)	JSON Lines	Stockage persistant et historique local des logs du simulateur.	Écriture en mode Append non bloquante pour préserver le thread principal.

5. Registre des Risques et Impacts Analytiques

ID	Description du Risque	Probabilité	Impact	Niveau	Stratégie de Mitigation & Logique Implémentée
R-01	Collision / Deadlock dans la ZONE X : Accès simultané à l'intersection critique.	Faible	Critique	Moyen	Verrou d'exclusion mutuelle (Mutex). Statut WAIT forcé pour le second agent jusqu'à libération complète.
R-02	Interruption Wi-Fi / Perte réseau : Coupure réseau en zone blanche industrielle.	Moyenne	Mineur	Faible	Edge Computing : Trajectoire locale maintenue de manière autonome. Reconnexion automatique en moins de 0.5s.
R-03	Surchauffe thermique des moteurs : Fonctionnement continu prolongé sous charge.	Basse	Majeur	Moyen	Surveillance de variable <code>temperature_c</code> . Limitation à 48.5°C avec refroidissement passif lors des arrêts.
R-04	Panne d'énergie en mission : Décharge complète sur une voie principale.	Faible	Majeur	Moyen	Seuil d'alerte configuré à 20%. Routage prioritaire automatique vers la ZONE R dès la fin de la mission active.

6. Fiche de Recommandations Priorisées

P1 : Optimisation de la Sécurité et Gestion du Trafic (Priorité Haute)

Action : Implémenter un système de détection d'obstacles dynamique à géométrie variable (zone tampon ajustable selon la vitesse).

Objectif : Réduire la distance de sécurité de 80 cm lors des approches à basse vitesse pour optimiser l'efficacité globale du flux sans risquer de collision.

Indicateur de succès : Diminution de 15% du temps d'attente cumulé en état STOP.

P2 : Résilience Réseau & Gestion des Données (Priorité Moyenne)

Action : Ajouter un mécanisme de mise en mémoire tampon locale (Local Buffering) dans `telemetry_sender.py` en cas de déconnexion MQTT prolongée.

Objectif : Éviter la perte de données de télémétrie lorsque le statut de connectivité passe à OFFLINE. Les données stockées en cache seront transmises en rafale dès le rétablissement de la liaison.

Indicateur de succès : Zéro perte de paquets JSON lors des tests de coupure réseau supérieure à 5 secondes.

P3 : Amélioration de l'IHM et Analytique (Priorité Basse)

Action : Intégrer des graphiques de tendances historiques (évolution de la batterie et de la température) directement sous le Canvas 3D.

Objectif : Permettre aux opérateurs de visualiser les cycles de dégradation énergétique sans consulter le fichier `telemetry_logs.json`.

Indicateur de succès : Visualisation claire des courbes de charge/décharge synchronisée à 10 Hz.

7. Guide d'Installation et d'Exécution

Assurez-vous d'avoir Python 3.9+ et Node.js (v18+) installés.

A. Installer les dépendances Python

Exécutez la commande suivante à la racine du projet :

```
pip install -r requirements.txt
```

B. Installer les dépendances Frontend

Basculez dans le dossier de l'interface et installez les paquets :

```
cd frontend  
npm install
```


C. Lancement des Services

Étape 1 : Activer le récepteur HTTP Webhook (Optionnel)

```
python3 webhook_server.py
```

Étape 2 : Lancer le Serveur WebSocket Headless (Moteur physique)

```
python3 headless_simulator.py
```

Étape 3 : Lancer l'Interface Web 3D

```
cd frontend  
npm run dev
```

Étape 4 : Ouvrir le Navigateur

Rendez-vous sur <http://localhost:5173/> pour manipuler la flotte d'AGV en temps réel.

8. Schéma de Données de Télémétrie (JSON)

Chaque trame de télémétrie respecte rigoureusement la structure suivante :

```
{  
  "agv_id": "AGV-01",  
  "state": "EN_ROUTE",  
  "battery_pct": 98,  
  "mission_id": "M-2026-06-01-01-04",  
  "speed_mps": 1.00,  
  "position": {  
    "x": 2.45,  
    "y": 1.50,  
    "zone": "A"  
  },  
  "target_zone": "C",  
  "distance_front_cm": 320,  
  "temperature_c": 26.4,  
  "connectivity_status": "ONLINE"  
}
```

9. Annexes et Ressources Médias

A. Liens des Fichiers et Code Source

- **Fichiers sources** : `warehouse_map.py`, `agv_agent.py`, `telemetry_sender.py`, `simulator.py`
- **Données de Télémétrie (Logs)** : `telemetry_logs.json` (Historique des payloads)
- **Projet complet** : [Dépôt GitHub Physical AI](#)

B. Démonstration Vidéo du Jumeau Numérique

- **Démonstration vidéo** : `Screen Recording 2026-06-01 at 15.25.11.mov` ([Lien Google Drive](#))